

Implementación de un Servo-motor de Alta Precisión mediante STM32 y FreeRTOS con Arquitectura Modular Orientada a Tareas

Informe de Avance de Proyecto — Modalidad Defensa de Proyecto

Ricardo Jeiel Arguello

1. Título y Resumen

Título: Implementación de un Servo-motor de alta precisión mediante STM32 y FreeRTOS con arquitectura modular orientada a tareas.

Resumen: El sistema implementa un servo-motor de posición de lazo cerrado sobre un microcontrolador STM32 (firmware portable entre STM32F103 y STM32F411), utilizando un motor paso a paso gobernado por un driver DRV8825 en configuración de 1/2 paso y un encoder magnético absoluto AS5600 (12 bits) como sensor de realimentación. La arquitectura, sobre FreeRTOS, distribuye el trabajo en seis tareas: adquisición del sensor por I2C con DMA, cálculo de control PID en punto fijo Q16, generación de movimiento sobre el driver, recepción y envío de telemetría por USB-HID, y supervisión del sistema. Durante esta etapa se resolvió un problema de oscilación sostenida cerca del punto de consigna, identificando que su causa raíz era la resolución mecánica mínima del motor (0.9° por paso en 1/2 step), mayor que la tolerancia que el lazo de control intentaba alcanzar. La solución combinó un filtro pasabajos exponencial sobre la posición leída, una zona muerta (deadband) en el cálculo del PID y la corrección de un error de redondeo en la conversión de ticks a grados.

2. Objetivos del Proyecto

Objetivo General

Implementar un servo-motor de posición basado en un motor paso a paso, controlado mediante una interfaz de consola USB-HID, que garantice precisión mecánica y estabilidad dinámica mediante el cumplimiento estricto de plazos de ejecución en un entorno de tiempo real (FreeRTOS).

Objetivos Específicos (Metas Medibles)

- **Determinismo en la Adquisición:** garantizar que la lectura del sensor AS5600 se realice de forma periódica y determinística mediante DMA, eliminando el bloqueo de CPU durante la transferencia I2C.
- **Precisión del Control:** lograr un error de estado estacionario acotado por la resolución mecánica del motor (0.9° por paso en 1/2 step), evitando oscilación sostenida alrededor del setpoint.
- **Latencia de Respuesta:** asegurar que el tiempo entre la llegada de un comando USB-HID y la actualización del movimiento del motor se mantenga acotado y consistente.
- **Eficiencia de Concurrencia:** mantener el lazo de control PID operando a frecuencia constante (ciclo de 3 ms) sin que la carga de comunicación USB o las tareas de monitoreo provoquen incumplimientos de plazo.
- **Estabilidad Dinámica:** eliminar la oscilación sostenida observada en regiones específicas del recorrido mediante el filtrado de la señal de posición y la introducción de una zona muerta dimensionada según la resolución física del actuador.

- Seguridad y Supervisión: detectar una condición de bloqueo mecánico (stall) en un tiempo acotado y proceder al apagado seguro del driver junto con la señalización visual de error.

Nota sobre la actualización de objetivos: respecto al preinforme, el objetivo de "Optimización Dinámica" (conmutación automática entre paso completo, 1/2 y 1/4 de paso según el error) finalmente no se implementó, tras evaluar que no aporta una mejora perceptible en este sistema: la resolución angular del encoder AS5600 (12 bits, 4096 posiciones por vuelta) es muchísimo más fina que la resolución mecánica del motor en cualquiera de sus configuraciones de paso, por lo que reducir el tamaño de paso cerca del setpoint no se traduce en una diferencia de precisión visible respecto a la que ya se logra con el deadband dimensionado según el paso de 1/2 step. El objetivo de muestreo a 1 ms por hardware puro también se redefinió: se optó por un software timer de FreeRTOS en lugar de un disparo por TIM1, ya que los timers de hardware del STM32 están pensados para resoluciones de tiempo mucho más finas (microsegundos), mientras que el período mínimo de muestreo viable del sistema resultó ser de 3 ms; para tiempos de ese orden, un software timer cumple el mismo propósito sin necesidad de reservar un periférico de hardware dedicado. Ambos cambios se detallan con mayor profundidad en la Sección 4.

3. Descripción Funcional del Sistema

El sistema opera como un servo-motor de lazo cerrado. La PC envía una consigna mediante USB-HID; el sistema adquiere la posición real vía el sensor magnético AS5600, calcula el error respecto al setpoint, procesa dicho error mediante un algoritmo PID en punto fijo Q16 y actúa sobre el driver DRV8825 mediante señales de pulso (STEP), dirección (DIR) y configuración de pasos (M0-M2).

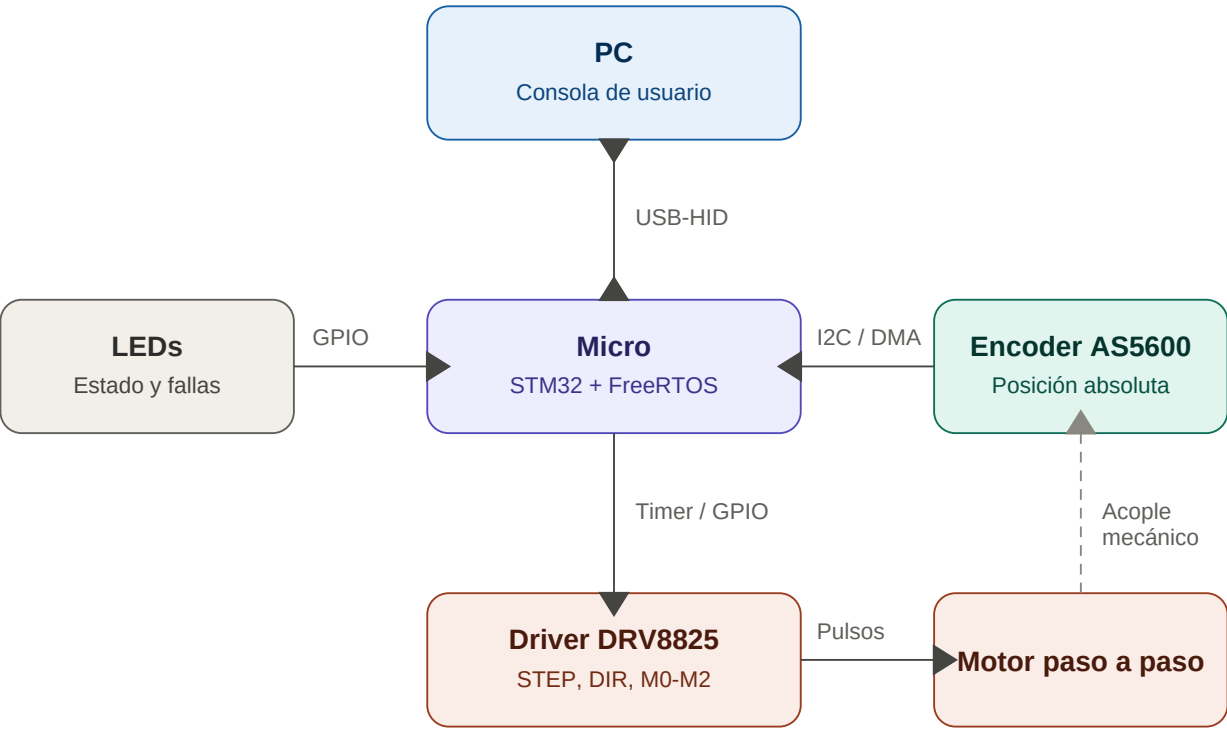
Entradas:

- Sensor AS5600 — posición angular absoluta vía bus I2C, con transferencia por DMA.
- Consigna de usuario — setpoint de posición en grados, enviado por USB-HID (OUT report).

Salidas:

- Driver DRV8825 — pulsos STEP generados por Timer (PWM con CCR fijo en 10), dirección por DIR, resolución de microstepping fija en 1/2 paso vía M0-M2.
- Telemetría — posición absoluta en grados, velocidad en RPM, cantidad de rotaciones y status del sistema, enviados por USB-HID (IN report).
- LEDs de estado — señalización de actividad y, en etapas futuras, de condición de falla (stall).

Diagrama de bloques general:



Línea sólida: señal eléctrica / comunicación. Línea punteada: acople mecánico.

4. Arquitectura de Software y Diseño en FreeRTOS

Esta sección refleja el estado real de la implementación a la fecha, que presenta diferencias respecto al diseño preliminar del preinforme, principalmente en el esquema de adquisición del sensor y en el mecanismo de sincronización entre tareas.

Identificación de Tareas

Tarea	Función	Prioridad
Driver_Task	Conversión de RPM a registro ARR del Timer; manejo de DIR y pulsos STEP.	7
Control_Task	Cálculo del error, PID en punto fijo Q16 con deadband, traducción a RPM objetivo.	6
(Timer Daemon Task — FreeRTOS)	Tarea interna del kernel; ejecuta el callback del software timer de 3 ms que dispara la transferencia DMA del sensor. No es una tarea propia del proyecto.	5
Sensor_Task	Lectura del AS5600 vía I2C/DMA; filtrado exponencial de la posición; publicación del dato a través de mutex.	4
Input_HID_Task	Recepción de setpoints de posición desde la PC vía USB-HID.	3
Output_HID_Task	Empaquetado y envío de telemetría hacia la PC vía USB-HID.	2
Monitor_Task	Supervisión de stack, CPU libre, heartbeat de LEDs y manejo de ENABLE.	1

Asignación de Prioridades

En una primera versión del firmware, las prioridades se asignaron de forma intuitiva, dando mayor jerarquía a las tareas vinculadas directamente al hardware del lazo de control y de comunicación (Sensor_Task, Driver_Task e Input_HID_Task, las tres en prioridad 4), una jerarquía intermedia a Control_Task y Output_HID_Task (prioridad 3) y la prioridad más baja a Monitor_Task (prioridad 2). Esta configuración presentaba una inversión de prioridad funcional: Control_Task, siendo el eslabón intermedio del pipeline crítico Sensor → Control → Driver, tenía una prioridad (3) inferior a la de Sensor_Task e Input_HID_Task (4), lo cual no garantizaba que el cálculo del PID se ejecutara de forma inmediata tras recibir el dato fresco del sensor.

Para corregir esta discrepancia se revisó la jerarquía completa, incorporando un factor que no había sido considerado originalmente: la Timer Daemon Task propia de FreeRTOS, configurada en este proyecto con prioridad 5 (configTIMER_TASK_PRIORITY), es la que efectivamente ejecuta el callback del software timer de 3 ms que dispara la transferencia DMA del sensor. Esto significa que cualquier tarea del proyecto con prioridad mayor a 5 puede, en el peor caso, retrasar levemente el disparo del siguiente ciclo de adquisición si está corriendo en el instante en que el timer debería vencer. Se midió que el tiempo de cómputo conjunto de Control_Task y Driver_Task es significativo respecto al período de 3 ms (un intento previo de reducir el período a 1 ms resultó en incumplimiento de plazos, confirmando que el presupuesto de tiempo de estas dos tareas no es despreciable). Con ese antecedente, se priorizó que estas dos tareas no fueran interrumpidas a mitad de su ejecución por la Daemon Task, ya que una preemption a mitad de la escritura de los registros del Timer (ARR/DIR) es más costosa para la estabilidad mecánica que un atraso de microsegundos en el disparo del muestreo.

La jerarquía final adoptada es: Driver_Task (7) > Control_Task (6) > Timer Daemon Task de FreeRTOS (5, fija) > Sensor_Task (4) > Input_HID_Task (3) > Output_HID_Task (2) > Monitor_Task (1). Driver_Task recibió la prioridad más alta por ser el último eslabón del pipeline, responsable de la actualización física del hardware una vez que Control_Task ya calculó el comando. Sensor_Task se ubicó por debajo de la Daemon Task, ya que su disparo real depende de la interrupción de hardware "DMA Transfer Complete" (inmune a las prioridades de FreeRTOS) y su carga de trabajo es liviana, por lo que no necesita preceder a la Daemon Task para cumplir su función. Input_HID_Task se reubicó de prioridad 4 a 3: se verificó mediante prueba directa, enviando comandos de cambio de setpoint mientras el motor estaba en movimiento activo, que esta reducción de prioridad no introduce un retardo perceptible en la respuesta del sistema a los comandos del usuario. Esta reasignación se realizó con margen, dado que configMAX_PRIORITIES está configurado en 56, dejando espacio para incorporar nuevas tareas en el futuro sin necesidad de reordenar la jerarquía existente.

Sincronización y Comunicación (IPC)

- Notificaciones de tarea (xTaskNotifyFromISR): utilizadas tanto para despertar a Sensor_Task al completarse la transferencia DMA del I2C, como para despertar a Input_HID_Task al llegar un nuevo reporte HID. Se adoptó este mecanismo en reemplazo de los semáforos binarios planteados originalmente, por su menor sobrecarga al no requerir una estructura de semáforo independiente.
- Mutex: protege la copia de los datos del sensor (estructura compartida de posición/ángulo/vueltas) entre Sensor_Task y las tareas consumidoras, evitando lecturas inconsistentes durante la actualización del dato filtrado.

- Colas (Queues): se implementaron cuatro colas con propósitos específicos. Queue1_ComHandle (tipo int32_t) transporta el setpoint de posición desde Input_HID_Task hacia Control_Task. Queue2_SensorControl (tipo EncoderData_t) transporta el dato completo del sensor desde Sensor_Task hacia Control_Task. Queue4_SensorOutputHID (tipo EncoderData_t) transporta ese mismo tipo de dato desde Sensor_Task hacia Output_HID_Task, para alimentar la telemetría. Queue3_PosHandle (tipo int32_t) transporta el comando calculado por Control_Task hacia Driver_Task.

Manejo de Interrupciones (ISRs)

El evento de adquisición del sensor está gobernado por un software timer de FreeRTOS configurado a un período de 3 ms (en lugar del disparo por hardware TIM1 a 1 ms planteado en el preinforme). Al vencer dicho timer, se inicia la transferencia DMA del bus I2C hacia el AS5600. La interrupción de "DMA Transfer Complete" es la que efectivamente notifica a Sensor_Task (vía xTaskNotifyFromISR), momento en el cual la CPU retoma el dato ya disponible en RAM sin haber estado bloqueada durante la transferencia. Las interrupciones del endpoint USB (recepción de OUT report) notifican de forma análoga a Input_HID_Task. Este cambio de TIM1+DMA a software-timer+DMA simplifica la configuración de timers de hardware a costa de un período de muestreo mayor (3 ms en lugar de 1 ms), lo cual fue aceptado como una decisión de diseño consciente dado que el período de 3 ms ya viene siendo utilizado de forma consistente en el resto del pipeline de control.

5. Grado de Avance

¿Se finalizó el desarrollo completo?

Sí. El lazo de control cerrado (Sensor → Control → Driver) está implementado, integrado y validado experimentalmente, incluyendo la resolución de los problemas de oscilación sostenida identificados durante la sintonización, la comunicación bidireccional por USB-HID con la PC, y la supervisión del sistema mediante Monitor_Task. Queda como mejora opcional, no implementada por restricciones de tiempo, la lógica automática de detección de bloqueo mecánico (stall): el pin de ENABLE ya está mapeado y Monitor_Task ya cuenta con un manejador de errores capaz de desenergizar el motor y señalar por LED qué tarea originó una falla, por lo que la incorporación de la detección de stall específica sobre esta base es directa de implementar en una etapa posterior. La lógica de microstepping dinámico (conmutación entre 1/2 y 1/4 de paso según el error) finalmente se descartó: la resolución del encoder es ampliamente mayor a la resolución mecánica del motor en cualquier configuración de paso, por lo que conmutar a pasos más finos cerca del setpoint no aporta una mejora de precisión perceptible frente a la ya lograda con la configuración fija de 1/2 paso y el deadband implementado.

Funcionamiento correcto de los periféricos

- I2C + DMA (AS5600): probado y funcional. La lectura periódica se realiza sin bloquear la CPU; se validó además un filtro pasabajos exponencial sobre la posición leída para atenuar el ruido de cuantización del sensor (12 bits, 4096 posiciones por vuelta).
- Timer de generación de pasos (STEP): probado. Configurado en modo PWM con CCR fijo en 10 y ARR variable según la velocidad calculada por el PID; se validó el movimiento del motor en 1/2 paso a distintas velocidades.
- GPIO de dirección (DIR) y microstepping (M0-M2): probados en su configuración fija de 1/2 paso; la conmutación dinámica entre resoluciones aún no fue implementada ni probada.

- USB-HID (Input y Output): probado de forma bidireccional. Se validó el envío de setpoints de posición desde la PC hacia el sistema, y la recepción de telemetría (posición absoluta, velocidad, rotaciones y status) en tiempo real desde el sistema hacia la PC.
- GPIO de ENABLE del driver: el pin está mapeado y gestionado por Monitor_Task, que implementa un manejador de errores (error handler) capaz de desenergizar el motor y señalar mediante LEDs cuál tarea originó la falla, facilitando además el debugueo. La lógica específica de detección automática de bloqueo mecánico (stall) que dispare este manejador no fue implementada por restricciones de tiempo; se documenta como mejora opcional en la Sección 5.

Modelado sobre RTOS

Las pruebas se realizaron en dos etapas. Primero, cada tarea se validó de forma individual mediante depuración (debugging) paso a paso, verificando su comportamiento aislado antes de integrarla al resto del sistema. Luego, el conjunto de las seis tareas se validó corriendo de forma concurrente sobre FreeRTOS, con sus prioridades y mecanismos de sincronización actuales (notificaciones de tarea y mutex), utilizando estructuras de depuración desarrolladas específicamente para este propósito (motor_debug, timer_debug, debug_stats), que permiten observar en tiempo real el estado del motor, los timers y estadísticas generales del sistema mientras las tareas operan en conjunto. El diagnóstico y la corrección del problema de oscilación sostenida descrito a continuación se realizaron sobre este esquema integrado, y no sobre un prototipo de lazo abierto o de un solo hilo de ejecución.

Resumen de hallazgos técnicos resueltos en esta etapa

- Pérdida de torque por microstepping: confirmado que 1/8 de paso producía pérdida de pasos a alta velocidad por caída de torque incremental; se consolidó la operación en 1/2 paso.
- Bache de posición (~899°) por truncamiento direccional: identificado como un error de redondeo en la conversión de ticks a grados, que truncaba siempre hacia valores crecientes en lugar de aplicar redondeo simétrico respecto al signo. En setpoints positivos este sesgo no era perceptible, pero en setpoints negativos generaba un offset sistemático que el sistema interpretaba como posición alcanzada sin estarlo realmente; al desplazar el eje manualmente, el lazo lo recapturaba en ese mismo punto sesgado. Se corrigió aplicando redondeo simétrico en dicha conversión.
- Oscilación sostenida en el setpoint (oscilación en 178° y -460°, amplitud $\approx 1^\circ$, velocidad $\approx 5.5^\circ/\text{s}$): diagnosticado como consecuencia de pedir al lazo una tolerancia de posición más fina que la resolución mecánica mínima del motor (0.9° por paso en 1/2 step, verificado en un motor de 200 pasos por vuelta en paso completo). Se corrigió mediante tres acciones: (1) corrección de un error de truncamiento en la conversión de ticks a grados, reemplazado por redondeo simétrico; (2) introducción de un filtro pasabajos exponencial sobre la posición leída en Sensor_Task; (3) introducción de una zona muerta (deadband) en Control_Task, dimensionada en al menos medio paso físico del motor, que anula la salida del PID cuando el error es menor a dicho umbral. Se descartó la hipótesis inicial de filtrar también el término derivativo del PID, ya que filtrar la señal en dos puntos distintos de la cadena (posición y derivada) introducía un retraso acumulado que desplazaba la oscilación a otras zonas del recorrido en lugar de eliminarla.